Heart Attack Risk Prediction Using Logistic Regression

COURSE NAME: Machine learning and Smart Systems A+B

Name: Beatrice Raphael Msokwa (Matr :38548252)

CODE DOCUMENTATION.

Introduction

Heart disease is one of the leading causes of mortality worldwide. Early prediction of heart attack risk can help improve medical decision-making and preventive care. In this project, we use machine learning techniques to analyze a dataset containing 8,763 patient records with 26 features, including:

- Demographics (Age, Sex, Country, Continent)
- Lifestyle factors (Smoking, Alcohol Consumption, Exercise Hours, Sleep Hours)
- Medical indicators (Cholesterol, Blood Pressure, Heart Rate, BMI, Diabetes)
- Target variable: Heart Attack Risk (0 = No risk, 1 = Risk)

The goal is to build a model that predicts whether a patient is at risk of a heart attack.

- What: Implementation of a binary logistic regression classifier to predict heart attack risk using patient medical records from Kaggle(8,763 rows.26 Features).
- Why: cardiovascular disease is a leading global cause of mortality. Early computational screening empowers preventive medicine and clinical decision making
- How: loaded the data using **pandas ,** drop unique identifiers(patient ID, Name), One hot encoded categorical variable using **pd.get_dummies()** And set Heart Attack Risk as the binary target
- Result: Established a clean, Structured data frame ready for machine learning pipelines.

Step-by-step implementation

Step 1: Data set exploration

- What: Inspect data set dimensions, Feature data types , Missing values and class distributions.
- Why: to understand data structures identify categorical vs. Numerical features and detect class imbalances that could bias predictions
- import pandas as pd
- How

```
import pandas as pd


df = pd.read_csv("heart_attack_prediction_dataset.csv")

print(df.shape)  # Output: (8763, 26)

    print(df.shape)  # Output: (8763, 26)
```

Results : confirmed 8763 patient recorded and 26 attributes , Noting a natural imbalance Between positive and negative heart attack outcomes

Step 2:Data preprocessing strategy

What; drop unnecessary identifiers columns, Converted categorical text into numerical columns via one hot encoding, and standardized numerical features using ‘standard scalar’.

Why: machine learning models require Purely numerical inputs. Scaring prevents features with large numerical ranges (e.g cholesterol levels) from overpowering small ones (e.g age), Avoiding gradient convergence warning

How:

``` python df = df.drop(columns=["Patient ID", "Name"], errors="ignore") df = pd.get_dummies(df, drop_first=True) X = df.drop(columns=["Heart Attack Risk"]) y = df["Heart Attack Risk"]

Results: Transformed all text attributes into binary columns and separate features (X) from the target label  (Y)
```

Step 3: train_test split implementation

- What: divided the data set into an 80% training set and 20% testing set using stratified sampling.
- Why: stratification(stratify=y) ensure that the ratio of heart attack cases is identical in both training and testing sets, preventing biased evaluation
- How
  from sklearn.model_selection import train_test_split

  X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.2, random_state=42, stratify=y
)

- Result: Create a reliable training and testing partitions (` 6,90-` training samples, ` 1,753` test samples).

Step 4: Logistic Regression Model Setup

- What: Configured the Logistic Regression classifier with the ` lbfgs` solver, L2 regularization (` C=1.0`), and an increased iteration limit (` max_iter=1000`).
- Why: Logistic regression models the probability $P(y=1\vert{}x) = \frac{1}{1 + e^{-z}}$ where $z = \beta_0 + \beta_1x_1 + \dots + \beta_nx_n$. Increasing ` max_iter` ensures the optimizer converges successfully on scaled data[cite: 7].
- How:
  ```python
  from sklearn.linear_model import LogisticRegression
  from sklearn.preprocessing import StandardScaler

  scaler = StandardScaler()
  X_train_scaled = scaler.fit_transform(X_train)
  X_test_scaled = scaler.transform(X_test)

  model = LogisticRegression(solver="lbfgs", C=1.0, max_iter=1000,
  random_state=42)

- Results: Initiatialize a robust, child optimization pipeline ready for gradient fitting

## Step 5: Model Training Process
  ```

- What: Fitted the logistic regression model on a scaled train data(X_train_scaled, y_train).
- Why: the optimization algorithm iteratively adjusts features coefficient (β) To minimize prediction error over the training set.
- How:

```
model.fit(X_train_scaled, y_train)
 print("Model trained successfully.")
```

- Result: The model successfully converged without warnings, producing trained coefficients for all predictor variables.

Step 6: Prediction Generation & Thresholding

- What: Generated continuous probability scores for test patients and applied a standard classification threshold (0.5).
- Why: Probabilities allow us to see how confident the model is before converting them into binary decisions (`0` for no risk, `1` for risk).
- How: ```python y_prob = model.predict_proba(X_test_scaled)[:, 1] y_pred = (y_prob >= 0.5).astype(int)

- Result: Produce discrete prediction levels (y_pred) for every record in the test set.

Step 7 performance evaluation implementation

- What|: evaluated model prediction using accuracy ,precision, recall F1 score and the confusion matrix

- Why: accuracy alone can be misleading on imbalanced medical data. Precision and recall give insight in false positive and false negative.

- How:

```
from sklearn.metrics import (
   accuracy_score,
   confusion_matrix,
   f1_score,
   precision_score,
   recall_score,
)

print("Accuracy:  ", accuracy_score(y_test, y_pred))
```

print("Precision: ", precision_score(y_test, y_pred, zero_division=0))
print("Recall: ", recall_score(y_test, y_pred, zero_division=0))
print("F1-Score: ", f1_score(y_test, y_pred, zero_division=0))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

- Result: Generated complete performance metrics and confusion matrix values.

Step 8: Results Interpretation & Visualization

- What: Plotted a visual confusion matrix heatmap using `matplotlib` and evaluated model coefficients.
- Why: Visual grids make it immediately clear how many true negatives, false positives, false negatives, and true positives occurred.
- How:
  ```python
  import matplotlib.pyplot as plt
  import numpy as np

  cm = confusion_matrix(y_test, y_pred)
  plt.figure(figsize=(5, 4))
  plt.imshow(cm, interpolation="nearest", cmap=plt.cm.Blues)
  plt.title("Confusion Matrix - Heart Risk Prediction")
  plt.colorbar() plt.show()
  ```

- Result: Visualized prediction performance clearly via a clean confusion matrix chart.